SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: CSA1407 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 10%

QUESTIONS: **Total Marks:50**

Annexure A

SCENARIO BASED TASK

Code of Conduct:

I P.Karpaga shree(192424042) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
P Karpaga shree

RUBRICS FOR CALCULATION:

Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and

Name: P. Roopaga Shree

Subject name: Compiler Design

Register number: 192421040142

Assessment: CO2-AT3

Subject code: CSA11407

Date: 06/08/2026

CO2-AT3 - Scenario Based Task

① Scenario: parsing Table conflict in LL(1)

An LL(1) parser constructs its parsing table using the FIRST and FOLLOW sets.

* The FIRST set contains all terminal symbols that can appear at the beginning of strings derived from a production.

* The FOLLOW set contains terminal symbols that can appear immediately after a non-terminal in a valid sentences.

During parsing table construction, every production is entered into the table according to its FIRST and FOLLOW sets.

A conflict occurs when more than one production is entered into the same parsing table cell for a non-terminal and an input symbol. This usually happens because the grammar contains:

* Left recursion
* common prefix (left factoring problem).
* Ambiguity

Because LL(1) parsing is deterministic, each table cell must contain only one production. If multiple

production. If multiple productions appear in the same call, the parser cannot decide which one to apply, leading to parsing errors or parsing failure.

Resolution:

To make the grammar suitable for LL(1) parsing:

* Eliminate left recursion
* Apply left factoring.
* Ensure that the FIRST and FOLLOW sets do not overlap.

These modifications produce a conflict free parsing table and enable deterministic parsing.

② Scenario: Incorrect parse tree Generation

A syntax analyzer constructs a parse tree by applying grammar productions to the sequence of input tokens.

For the expression:

id + id - id

the parse tree determines the order in which operations are evaluated.

If the grammar does not specify operator associativity, the parser may generate the expression as:

id + (id - id)

Instead of

(id + id) - id

This incorrect grouping changes the evaluation order and may produce incorrect results.

Grammar rules directly determines the structure of the parse tree. If associativity is not enforce - mental, multiple valid parse trees can be generated for the same expression.

Decision:

The grammar should enforce left associativity for addition and subtraction.

Example grammar:

$$E \rightarrow E+T \mid E-T \mid T$$

$$T \rightarrow id$$

- This grammar ensures that expressions are evaluated from left to right and produces the correct parse tree for accurate evaluation.

③ Scenario: Invalid Expression Handling

The Lexical analyzer correctly recognized the tokens of the expression:

a + * b

The tokens (a, +, *, b) are individually invalid.

However, the Syntax analyzer check whether the tokens follow the grammar rules.

After the '+' operator, the grammar expects an operand, but another operator ('*') appears instead. Therefore, the token sequence violates the grammar,

making it impossible to generate a valid parse tree.

In predictive (LL) parsing, the parser detects the error when no valid production exists in the parsing table for the current input symbol.

In shift-reduce (LR) parsing, the parser detects the error when no valid shift or reduce action is available.

Without proper recovery techniques, the parser typically stops after reporting the syntax error.

Decision:

The parser should implement suitable error detection and recovery techniques such as:

* panic-mode recovery
* phrase-level recovery
* Synchronization using Follow sets

These techniques allow the parser to continue parsing the remaining input while providing meaningful error messages.

④ Scenario: Ambiguity in conditional statements

The Grammar:

$S \rightarrow$ if E then S

$S \rightarrow$ if E then S else S

$S \rightarrow a$

This grammar is ambiguous because the parser cannot determine which if statement an else belongs to.

This ambiguity is known as the dangling else problem.

Both productions begin with same prefix:

if E then S

Therefore, while constructing the parsing table, conflicts occurs because the parser cannot decide whether to use the first production or continue with the second production containing else.

This ambiguity introduces parsing conflicts and allows multiple valid parse trees for the same inputs.

Decision:

The grammar should be rewritten using matched and unmatched statements.

Example:

S → M | U

M → if E the M else M | a

U → if E then S | if E then M

else U

Alternatively, parse generators resolve the ambiguity by associating else with the nearest unmatched if, which is the standard rule used in most programming languages.

(5) Scenario: operator precedence conflict

The Grammar:

E → E+E

E→E * E

E→id

This grammar is ambiguity because it does not define operator precedence or associativity.

For the expression

id + id * id

two different parse trees are possible:

1. (id + id) * id

2. id + (id * id)

The correct evaluation is:

id + (id * id)

because multiplication has higher precedence than addition.

Since the grammar does not define precedence, the parser cannot determine the correct parse tree.

Decision:

The grammar should be rewritten as:

E → E+T | T

T → T * F | F

F → id

In this grammar:

* Multiplication has higher precedence than addition.

* Addition and multiplication are left-associative.

Alternatively, parse generators can use precedence and associativity declarations to resolve these conflicts automatically.